

Performance Analysis Report

Multi-Client Chat System

CS 3205 -- Computer Networks | CS23B038

1. Methodology

Servers tested

- * fork (port 9001) -- one child process per client. The parent routes messages through pipes and a select() loop.
 - * thread (port 9002) -- one thread per client. A global mutex protects the client table; a per-client mutex makes sure only one sender writes to that socket at a time.
 - * epoll (port 9003) -- single-threaded event loop using epoll (edge-triggered).
- All sockets are non-blocking.

All three use the same message format: 76-byte fixed header + variable payload (max 1024 bytes).

Load test

10 clients connect at the same time (held at a barrier) and each sends 20 broadcast messages, one every 5 ms. Latency is measured as the time from when a message is sent to when the echo arrives back on the same socket.

Stress test

Clients are added in steps: 2, 4, 6, 8, 10. At each step, new clients connect and start sending. The old clients keep running -- they are never disconnected. Each step runs for 6 seconds while CPU and memory are sampled every second.

Metrics collected

- * CPU% -- measured across the whole server process tree (including fork children).
- * VmRSS -- resident memory from /proc/{pid}/status, summed over all processes.
- * PSS -- proportional memory from /proc/{pid}/smaps_rollup, summed over all processes.
- * Latency -- time between send() and receiving the broadcast echo, in milliseconds.

2. Test Results

Load test (10 clients x 20 messages each)

Server	Msgs	Mean lat	P95 lat	CPU avg	VmRSS	PSS

Fork	200	2.15 ms	7.88 ms	2.0 %	5.0 MB	0.6 MB
Thread	200	1.07 ms	3.01 ms	1.3 %	1.6 MB	0.4 MB
Epoll	200	1.22 ms	3.43 ms	1.7 %	2.1 MB	0.8 MB

Thread had the lowest average latency (1.07 ms), followed closely by epoll (1.22 ms). Fork was slower (2.15 ms) because every message travels through a pipe to the parent before being forwarded -- thread and epoll write directly to sockets.

Stress test (cumulative ramp 2 -> 4 -> 6 -> 8 -> 10 clients)

Fork: CPU rose from 1.5% (2 clients) to 12.2% (10 clients).
VmRSS grew from 2.4 MB to 5.4 MB -- each new child adds about 0.75 MB.
Mean latency stayed low throughout: 0.14 -- 0.18 ms.

Thread: CPU rose from 0.7% to 4.5%. VmRSS stayed flat at 1.6 MB.
Mean latency: 0.12 -- 0.17 ms.

Epoll: CPU rose from 0.8% to 5.0%. VmRSS stayed flat at 2.1 MB.
Mean latency rose slightly at 10 clients: 0.14 -- 0.25 ms.

Message Delivery Time Distribution (Fork vs Threaded vs Epoll)

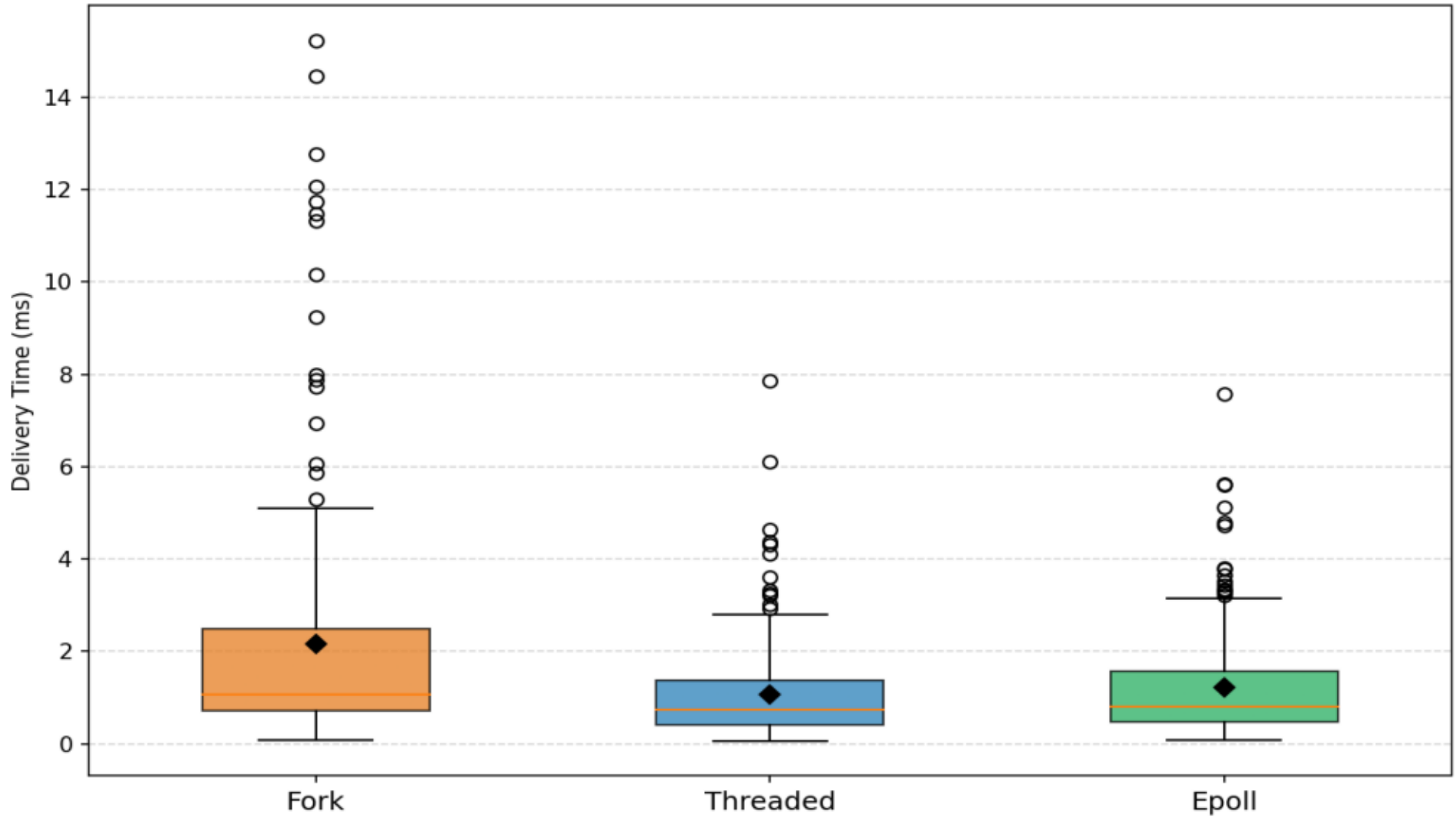


Figure 1 -- Message delivery time distribution (load test, 10 clients x 20 msgs)

CPU Usage vs Number of Clients (Fork / Threaded / Epoll)

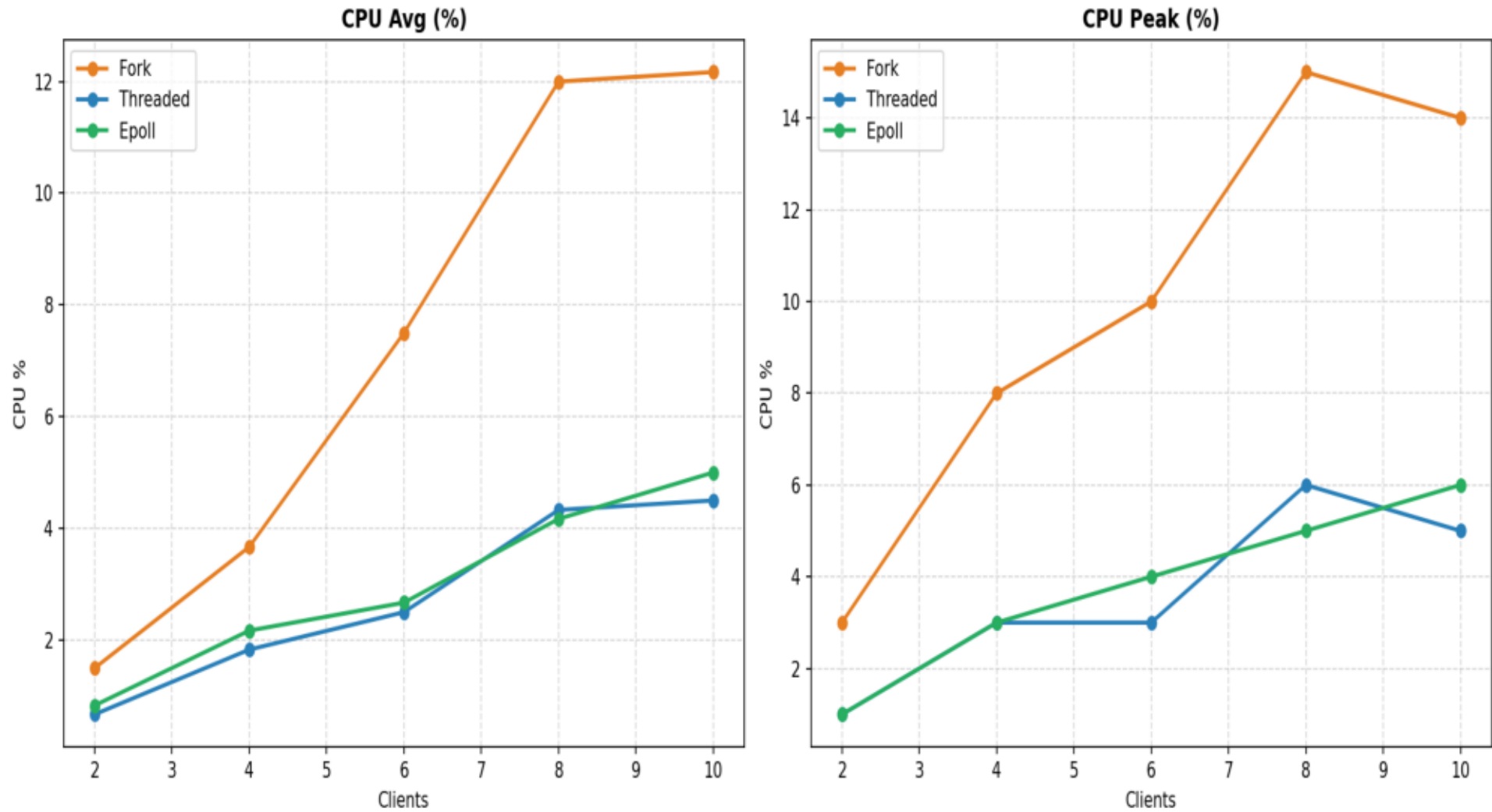


Figure 2 -- CPU average and peak usage vs number of connected clients (stress test)

Memory Usage vs Number of Clients (Fork / Threaded / Epoll)

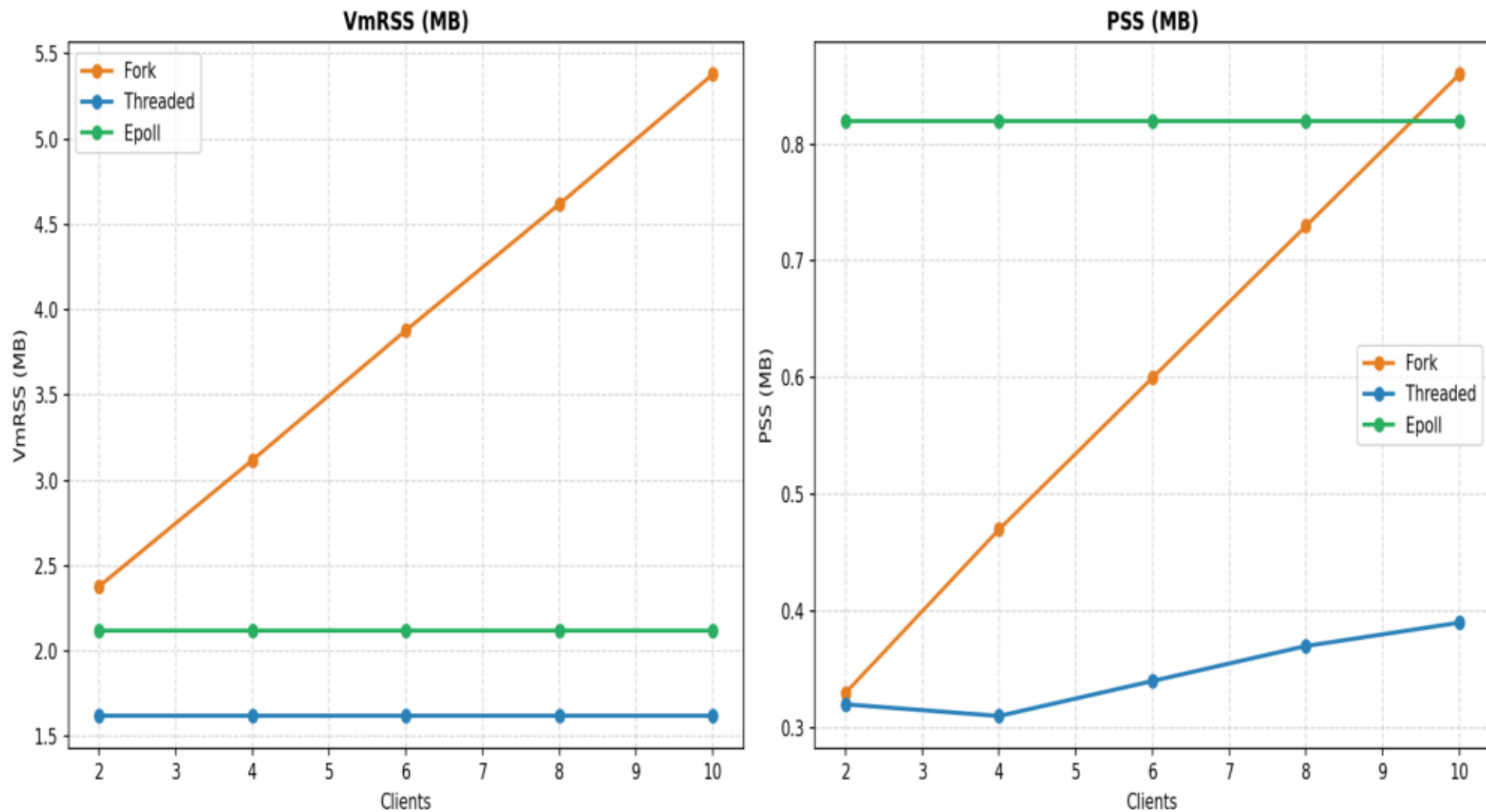


Figure 3 -- VmRSS and PSS memory vs number of connected clients (stress test)

**Message Delivery Time vs Clients
(Fork / Threaded / Epoll)**

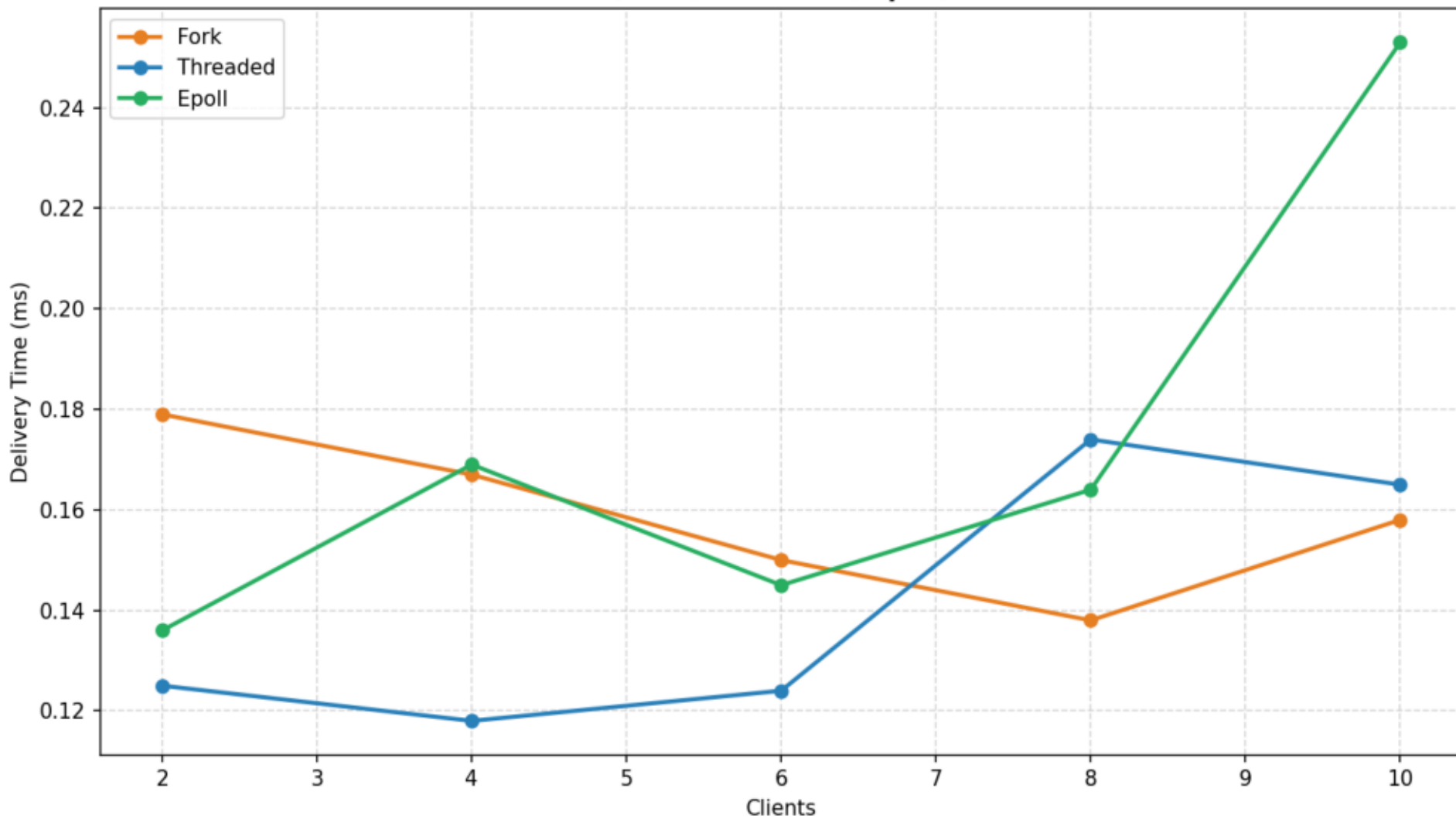


Figure 4 -- Average message delivery time vs number of connected clients (stress test)

3. Analysis and Observations

Latency in the load test

Thread and epoll are faster than fork because they write messages directly to the destination sockets. Fork has an extra step -- the child sends the message through a pipe to the parent, which then forwards it to all other clients. This adds roughly 1 ms of extra latency at 10 clients.

CPU usage grows fastest for fork

The parent process calls `select()` on all pipe file descriptors every loop, even when most clients are idle. This is why fork hits 12% CPU at 10 clients while thread and epoll stay under 5%. Thread and epoll only do work when a socket actually has data.

Memory grows with fork, stays flat for thread and epoll

Every new client in fork creates a new process, adding about 0.75 MB each. At 10 clients fork uses 5.4 MB. Thread and epoll run as a single process so their memory does not change: 1.6 MB for thread and 2.1 MB for epoll regardless of clients.

Stress test latency stays low for all three

All three servers kept delivery times well under 1 ms in the stress test across all client levels. None of them hit a hard bottleneck in the 2 to 10 client range. The small rise in epoll at 10 clients (0.25 ms) is the first sign of the event loop starting to feel the load.

4. Bottlenecks Identified

Fork -- messages travel through too many steps

Every broadcast message goes through 4 steps: child writes to pipe, parent reads it, parent writes to other pipes, other children read and send to sockets. At 10 clients this costs about 1 ms extra latency compared to the other two servers.

Also, the parent checks all the pipes in a loop even when clients are idle, which wastes CPU and gets worse as more clients connect.

Thread -- one lock per socket on every broadcast

To send a broadcast, the server locks each destination socket one at a time and sends to it before moving to the next. With 10 clients sending at the same time, these lock requests pile up and form a queue. This is what causes the P95 latency (3 ms) to be roughly 3x the mean (1 ms).

Epoll -- event loop blocks during broadcast writes

The event loop sends a broadcast by writing to 9 sockets one by one without handling any other events in between. If one write takes longer (e.g. socket buffer is full), all other incoming messages wait. This is visible as a latency increase at 10 clients.

5. Potential Optimizations

Fork -- replace select() with epoll in the parent

Instead of checking all pipe file descriptors on every loop, the parent would only wake up when a pipe actually has data. This would reduce idle CPU usage and keep performance better as client count grows.

Thread -- give each client its own message queue

Instead of locking and writing to each socket directly, broadcast could add messages into each client's own queue and return immediately. A dedicated sender thread per client sends from its own queue. This removes the need to wait for locks one at a time.

Epoll -- offload broadcast writes to worker threads

A small thread pool (2-4 workers) could handle the actual socket writes. The event loop just hands off the broadcast task and picks up the next event right away. This would keep the loop responsive under load.

All servers -- handle more than 10 clients gracefully

Currently the 11th connection is refused immediately. A proper waiting list would let the server hold new connections and serve them as slots open up.